

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ № 7

**«Использование паттерна «Итератор»,
паттерна «Фабричный метод» и
рефлексии в Java»**

по курсу
Объектно-ориентированное программирование

Выполнил: Волынец Никита
6203-010302D

Оглавление

Задание №1	3
Задание №2	5
Задание №3	6

Задание №1

Для реализации итераторов в TabulatedFunction добавлено наследование от Iterable<FunctionPoint>. В ArrayTabulatedFunction итератор работает с внутренним массивом точек,

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0; // текущий индекс для итерации 3 usages

        @Override
        public boolean hasNext() {
            return currentIndex < point_count; // проверяет есть ли следующая точка
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("No more points"); // исключение если точек нет
            }
            // создаем копию точки для защиты инкапсуляции
            FunctionPoint point = point_mass[currentIndex]; // получаем текущую точку
            FunctionPoint copy = new FunctionPoint(point.getX(), point.getY()); // создаем копию то
            currentIndex++; // увеличиваем индекс
            return copy; // возвращаем копию
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Remove operation is not supported"); // операц
        }
    };
}
```

в LinkedListTabulatedFunction - с узлами списка.

```

@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.getNext(); // начинаем с первого узла после головы
        private final FunctionNode endNode = head; // конечный узел - голова 1usage

        @Override
        public boolean hasNext() {
            return currentNode != endNode; // проверяем не дошли ли до конца
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("No more points"); // исключение если точек нет
            }
            // создаем копию точки для защиты инкапсуляции
            FunctionPoint point = currentNode.getPoint(); // получаем точку из текущего узла
            FunctionPoint copy = new FunctionPoint(point.getX(), point.getY()); // создаем копию т
            currentNode = currentNode.getNext(); // переходим к следующему узлу
            return copy; // возвращаем копию
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Remove operation is not supported"); // опера
        }
    };
}

```

Оба итератора возвращают копии точек для защиты инкапсуляции, метод `remove()` не поддерживается. В методе `main` проверяется работа через вывод всех точек функции в цикле `for-each`.

```

System.out.println("Task 1: "); // вывод заголовка задания 1

TabulatedFunction arrayFunc = new ArrayTabulatedFunction( leftX: 0, rightX: 15, pointsCount: 14);
for (FunctionPoint p:arrayFunc){ // цикл for-each по точкам функции
    System.out.println(p); // вывод текущей точки
}

```

```

Task 1:
(0.0,0.0)
(1.1538461538461537,0.0)
(2.3076923076923075,0.0)
(3.4615384615384612,0.0)
(4.615384615384615,0.0)
(5.769230769230768,0.0)
(6.9230769230769225,0.0)
(8.076923076923077,0.0)
(9.23076923076923,0.0)
(10.384615384615383,0.0)
(11.538461538461537,0.0)
(12.692307692307692,0.0)
(13.846153846153845,0.0)
(14.999999999999998,0.0)

```

Задание №2

Реализован паттерн "Фабричный метод" через интерфейс TabulatedFunctionFactory с тремя методами создания.

```
package functions;

public interface TabulatedFunctionFactory { 4 usages 2 implementations
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); // создает табулирован
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
}
```

Созданы фабрики ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory как вложенные классы.

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { 2 usages
    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points); // создает массивную табулированную функцию из точ
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount); // создает массивную функцию с
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values); // создает массивную функцию с гран
    }
}

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { 1 usage
    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points); // создает связную табулированную функцию из
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount); // создает связную функци
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values); // создает связную функцию с п
    }
}
```

В TabulatedFunctions добавлено статическое поле фабрики и метод setTabulatedFunctionFactory() для его замены. Все создания объектов заменены на вызовы методов фабрики.

```

private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); // фабрика

// метод для замены фабрики
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory) { 2 usages
    factory = newFactory; // устанавливаем новую фабрику
}

// три перегруженных метода создания табулированных функций
public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { no usages
    return factory.createTabulatedFunction(points); // создаем функцию через фабрику из точек
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usages
    return factory.createTabulatedFunction(leftX, rightX, pointsCount); // создаем функцию через фабрику с границами
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { no usages
    return factory.createTabulatedFunction(leftX, rightX, values); // создаем функцию через фабрику с границами и значениями
}

```

Проверка в main показывает смену типов объектов при изменении фабрики.

```

System.out.println("\nTask 2: "); // вывод заголовка задания 2
Function f = new Cos(); // создание функции косинуса
TabulatedFunction tf; // объявление переменной для табулированной функции

tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11); // табулирование функции с фабрикой по умолчанию
System.out.println(tf.getClass()); // вывод типа созданной функции

TabulatedFunctions.setTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11); // табулирование с новой фабрикой
System.out.println(tf.getClass()); // вывод типа созданной функции

TabulatedFunctions.setTabulatedFunctionFactory(new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory()); // установка фабрики массива
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11); // табулирование с фабрикой массива
System.out.println(tf.getClass()); // вывод типа созданной функции

```

Task 2:

```

class functions.ArrayTabulatedFunction
class functions.ArrayTabulatedFunction
class functions.ArrayTabulatedFunction

```

Задание №3

Добавлены методы createTabulatedFunction() с параметром Class, использующие рефлексию для создания объектов. Методы находят нужный конструктор через getConstructor() и создают экземпляр через newInstance(). При ошибках рефлексии выбрасывается IllegalArgumentException с причиной.

```

public static TabulatedFunction createTabulatedFunction(Class<?> functionClass, FunctionPoint[] points) { 1 usage
    // проверяем что класс реализует tabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Class must implement tabulatedFunction interface"); // исключение если класс не реализует
    }

    try {
        // ищем конструктор с параметром functionpoint[]
        Constructor<?> constructor = functionClass.getConstructor(FunctionPoint[].class); // получаем конструктор
        // создаем объект с помощью рефлексии
        return (TabulatedFunction) constructor.newInstance((Object) points); // создаем экземпляр через рефлексия
    } catch (Exception e) {
        throw new IllegalArgumentException("Error creating function with points array", e); // исключение при ошибке создания
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> functionClass, double leftX, double rightX, int pointsCount) { 1
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Class must implement tabulatedFunction interface"); // исключение если класс не реализует
    }

    try {
        // ищем конструктор с параметрами double double int
        Constructor<?> constructor = functionClass.getConstructor(double.class, double.class, int.class); // получаем конструктор
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, pointsCount); // создаем экземпляр через рефлексия
    } catch (Exception e) {
        throw new IllegalArgumentException("Error creating function with bounds and point count", e); // исключение при ошибке создания
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> functionClass, double leftX, double rightX, double[] values) {
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Class must implement tabulatedFunction interface"); // исключение если класс не реализует
    }

    try {
        // ищем конструктор с параметрами double double double[]
        Constructor<?> constructor = functionClass.getConstructor(double.class, double.class, double[].class); // получаем конструктор
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, values); // создаем экземпляр через рефлексия
    } catch (Exception e) {
        throw new IllegalArgumentException("Error creating function with bounds and values", e); // исключение при ошибке создания
    }
}

public static TabulatedFunction tabulate(Class<?> functionClass, Function function, double leftX, double rightX, int pointsCount) {
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Class must implement tabulatedFunction interface"); // исключение если класс не реализует
    }

    if (function == null) {
        throw new IllegalArgumentException("Function cannot be null"); // проверяем что функция не null
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("The number of points must be at least 2"); // проверяем минимальное количество точек
    }
    if (leftX >= rightX) {
        throw new IllegalArgumentException("The left border must be smaller than the right one"); // проверяем порядок границ
    }
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Tabulation boundaries extend beyond the definition domain"); // проверяем что границы в
    }

    double vals[] = new double[pointsCount]; // создаем массив для значений
    double step = (rightX - leftX) / (pointsCount - 1); // вычисляем шаг

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step; // вычисляем x координату
        vals[i] = function.getFunctionValue(x); // вычисляем значение функции
    }

    return createTabulatedFunction(functionClass, leftX, rightX, vals); // создаем функцию через рефлексия
}

```

```

public static TabulatedFunction inputTabulatedFunction(Class<?> functionClass, InputStream in) throws IOException {
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) { // проверяем что класс реализует нужный интерфейс
        throw new IllegalArgumentException("Class must implement TabulatedFunction interface"); // бросаем исключение
    }

    if (in == null) { // проверяем что поток не нулевой
        throw new IllegalArgumentException("Input stream cannot be null"); // бросаем исключение если поток нулевой
    }

    DataInputStream dataIn = new DataInputStream(in); // создаем поток для чтения данных

    int pointsCount = dataIn.readInt(); // читаем количество точек
    FunctionPoint[] points = new FunctionPoint[pointsCount]; // создаем массив для точек

    for (int i = 0; i < pointsCount; i++) { // цикл по всем точкам
        double x = dataIn.readDouble(); // читаем координату x
        double y = dataIn.readDouble(); // читаем координату y
        points[i] = new FunctionPoint(x, y); // создаем новую точку
    }

    return createTabulatedFunction(functionClass, points); // создаем табулированную функцию через рефлексию
}

public static TabulatedFunction readTabulatedFunction(Class<?> functionClass, Reader in) throws IOException {
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) { // проверяем что класс реализует нужный интерфейс
        throw new IllegalArgumentException("Class must implement TabulatedFunction interface"); // бросаем исключение
    }

    if (in == null) { // проверяем что поток не нулевой
        throw new IllegalArgumentException("Reader cannot be null"); // бросаем исключение если поток нулевой
    }

    StreamTokenizer tokenizer = new StreamTokenizer(in); // создаем токенизатор для чтения
    tokenizer.parseNumbers(); // настраиваем обработку чисел как double

    if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) { // читаем следующий токен и проверяем что это число
        throw new IOException("Expected number of points"); // бросаем исключение если не число
    }

    int pointsCount = (int) tokenizer.nval; // получаем количество точек

    FunctionPoint[] points = new FunctionPoint[pointsCount]; // создаем массив для точек

    for (int i = 0; i < pointsCount; i++) { // цикл по всем точкам
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) { // читаем следующий токен и проверяем что это число
            throw new IOException("Expected x coordinate"); // бросаем исключение если не число
        }
        double x = tokenizer.nval; // получаем координату x

        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) { // читаем следующий токен и проверяем что это число
            throw new IOException("Expected y coordinate"); // бросаем исключение если не число
        }
        double y = tokenizer.nval; // получаем координату y

        points[i] = new FunctionPoint(x, y); // создаем новую точку
    }

    return createTabulatedFunction(functionClass, points); // создаем табулированную функцию через рефлексию
}

```

В main протестировано создание объектов разных типов через рефлексию с различными параметрами.


```

System.out.println("\nTask 3: "); // вывод заголовка задания 3
TabulatedFunction f2; // объявление переменной для табулированной функции

f2 = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3); // создани
System.out.println(f2.getClass()); // вывод типа созданной функции
System.out.println(f2); // вывод содержимого функции

f2 = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10}); //
System.out.println(f2.getClass()); // вывод типа созданной функции
System.out.println(f2); // вывод содержимого функции

f2 = TabulatedFunctions.createTabulatedFunction(LinkedListTabulatedFunction.class, // создание связанной функции через рефл
    new FunctionPoint[] { // массив точек
        new FunctionPoint(x: 0, y: 0), // точка начала
        new FunctionPoint(x: 10, y: 10) // точка конца
    }
);
System.out.println(f2.getClass()); // вывод типа созданной функции
System.out.println(f2); // вывод содержимого функции

f2 = TabulatedFunctions.tabulate(LinkedListTabulatedFunction.class, new Sin(), leftX: 0, Math.PI, pointsCount: 11); // табули
System.out.println(f2.getClass()); // вывод типа созданной функции
System.out.println(f2); // вывод содержимого функции

```

Task 3:

```

class functions.ArrayTabulatedFunction
{(0.0;0.0), (5.0;0.0), (10.0;0.0)}
class functions.ArrayTabulatedFunction
{(0.0;0.0), (10.0;10.0)}
class functions.LinkedListTabulatedFunction
{(0.0,0.0), (10.0,10.0)}
class functions.LinkedListTabulatedFunction
{(0.0,0.0), (0.3141592653589793,0.3090169943749474), (0.6283185307179586,

```